1. How would you represent the input of the network (vector, matrix, multiple inputs etc.)
2. How would you design the output layer
3. Which activation functions would you use
4. Which Loss function
5. Which (possible) weight regularizers, initializers, etc.
6. On which hyperparameters would you perform the model selection and why
7. How to evaluate the test performance

Introduction

- Which kind of task
    - Supervised/unsupervised
    - Binary/multi-class
    - Single label/multi label
- What we have to predict
- *“Find best architecture that is the least complex possible”*
- *“find best hyperparameters that allows the model to perform good both on training and on test”*
- *“the model should be a trade off between computational complexity and accuracy”*
- *“avoid overfitting”*

How would you represent the input of the network (vector, matrix, multiple inputs etc.)

- What is the input (also as vector)
    - Dimension of a single input
    - Dimension of the whole input
- What kind of model
    - Why CNN
    - Avoid MLP → Shift variance issue (example of dog)
        - Universal boolean function: can compute any boolean function
        - Universal classifiers: can capture any classification boundary
        - Universal approximators: one hidden layer is enough to approximate an arbitrary function, but deeper it is the less complex it becomes
    - Avoid RNN → Designed for sequential data of variable length (like text or audio)

Dimensionality reduction with autoencoders

Is the process of reducing the number of dimensions in the data by either excluding less useful features (feature selection) or transforming the data into lower dimensions (feature extraction). This allows us to prevent overfitting, that is when the model learns too well from the training dataset and fails to generalize well for unseen data.

Autoencoders are an unsupervised artificial neural network that attempts to encode the data by compressing it into lower dimensions and then decoding the data to reconstruct the image. The bottleneck layer holds the compressed representation of the input data. With autoencoders the number of output units must be equal to the number of input units since we're attempting to reconstruct the input data.
The number of neurons in the layer of the encoder will be decreasing as we move on with further layers, while in the decoder it will increase as we move on with further layers. As an example I could have 32, 16 and then 7 units in each layer of the encoder, and then 7, 16 and 32 in the decoder. Before feeding the data into the autoencoder we should scale it between 0 and 1 using MinMaxScaler.

When we use autoencoders for dimensionality reduction we'll be extracting the bottleneck layer located between the encoder and the decoder and use it to reduce the dimension, this process can be viewed as feature extraction. Furthermore, we will be using a deep autoencoder, that is when the encoder and decoder are symmetrical. The hyper partners of an autoencoder are:

- Number of units in the bottleneck layer
- Input and output size (number of features in the data)
- Number of neurons in each layer
- Number of layer sin encoder and decoder
- Activation function
- Optimization function

Augmentation

- Apply transformations to the images
    - **Rotations**
    - **Translations**
    - **Flips**: could be *horizontal* or *vertical* (or both)
    - **Zoom** (zooming in and out): usually -0.2 + 0.3 if we zoom out we need to specify how to cover the new space with MNIST dataset use black (that is the background colour)
- Reduce overfitting by increasing the test size

How would you design the output layer

- Number of neurons in output layer = number of classes
- Softmax activation function
    - Formula $f(z) = \frac{e^{z}}{\sum_{i=1}^{N} e^{z_i}}$

    - Single output: value between 0 and 1
    - All the outputs: summed up are = 1
- If binary classification task we can use the sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$ + graph
- If the classes are represent as a string we should first transform it to integers or one-hot vectors

Which activation functions would you use

- Output layer: **softmax**

- Activation functions for hidden layer in MLP, start with ReLU
    - **ReLU**
        - ReLU $f(z) = max(0, z)$ + graph
        - [0; +inf]
        - Most used for hidden layers
        - Easy to compute and quick
        - Has little vanishing gradient problem
        - Does not take into account negative values by mapping them to 0
    - **LeakyReLU**
        - $f(z) = max(az, z)$ where a is small number a = 0.01 + graph
        - [-inf; +inf]
        - no vanishing gradient problem
    - **Sigmoid**
        - $\sigma(z) = \frac{1}{1+e^{-z}}$ + graph
        - [0; 1]
        - Suffer from vanishing gradient problem
        - Use for binary classification problems
    - **Linear**
        - $f(z) = z$ + graph
        - [-inf; +inf]
    - **Tanh**
        - $f(z) = \frac{e^{z}-e^{-x}}{e^{z}+e^{-z}}$ + graph
        - [-1; +1]
        - Useful when negative values are desired
        - Zero-centered, so easier to optimize
        - Suffer from vanishing gradient problem) between
    - **Swish**
        - $f(z) = z\, sigmoid(\beta z)$ where β is a trainable parameter
        - Allows data normalization and quicker convergence
        - Works better with neural networks that require LSTM, compared with ReLu

➔ Additionally use a Grid Search to find the best activation function for the hidden layers in a MLP

- Activation functions for CNN (Convolutional layer + pooling layer)
  - By default set to None
  - Keep None
  - No activation function will be applied

Which Loss functions

- The loss function is the function that computes the distance between the current output of the model and the expected output
- It's a method to evaluate how your algorithm models the data
- Different loss function, depends on how labels are represented and on the task we are dealing with:
  - Regression task with many outliers ⟹ **mean absolute error,** since it is less sensitive to outliers (since it isn't squared, the outliers don't have too big weights) $MAE = \frac{1}{N}\sum_{i=1}^{N}\left|Y_i - \hat{Y}_i\right|$
  - Regression task ⟹ **mean square error**. Compute the average square difference between predicted values and true values $MSE = \frac{1}{N}\sum_{i=1}^{N}\left(Y_i - \hat{Y}_i\right)^2$
  - Binary classification ⟹ **binary cross entropy loss**. Measure dissimilarity between predicted and true binary labels

    $$BCE = -\frac{1}{n}\sum_{i=1}^{n}(Y_i * log\left(\hat{Y}_i\right) + \left(1 - Y_i\right) * log\left(1 - \hat{Y}_i\right))$$
  - Multiclass with output as **one-hot vector** ⟹ **categorical cross entropy loss**
  - Multiclass with output as **integer** ⟹ **sparse categorical cross entropy loss**
  - **Square hinge loss** ⟹ converge faster and better performance + more robust to noise in the training set labeling + quadratically increase with the error (→ larger errors are punished more heavily)

Which (possible) weight regularizers, initializers, etc.

Initializers

- Initialize **weights at random** can lead to:
  - Vanishing gradients: weights and gradients close to 0. Convergence to minimize the loss are very slow 🙁
  - Exploding gradients: weights and gradients greater than 1. Impossible to find the optimal minimum 🙁
- Initialize weights with random uniform or normal distribution (mean = 0 and standard deviation = 0.1)
- Xavier/Glorot initialization (**tanh or sigmoid**)

    - Goal: keep variance (and variance of each gradient) equal for each layer
    - Takes into account the number of inputs and outputs of a layer
    - Two types *uniform* and *normal*
        - Uniform: take the values of the weights with the same probability from the range $\left[-\sqrt{\frac{6}{input+output}}\ ;\ \sqrt{\frac{6}{input+output}}\right]$
        - Normal: take the values of the weights from a normal distribution with mean = 0 and standard deviation $\sigma = \sqrt{\frac{2}{input+output}}$
- He initialization (**ReLU**)
    - Small variation of Xavier initialization
    - Initialize weights from normal distribution
    - Scale weights proportionally to the number of inputs to the layer
    - Two types *uniform* and *normal*
        - Uniform: take the values of the weights with the same probability from the range $\left[-\sqrt{\frac{2}{input}}\ ;\ \sqrt{\frac{2}{input}}\right]$
        - Normal: the weights values are taken from a normal distribution with mean 0 and standard deviation equal to $\sigma = \sqrt{\frac{2}{input}}$

Regularizer

- Weight regularization
    - Add to the loss function of the model a cost associated with having large weights
    - Use to mitigate overfitting
    - Put constraint on the model
    - Force weight to take small values
    - Make distribution of weight more regular
- **L1**: cost added proportionally to the absolute value of the weight coefficients. Encourage the model to reduce the less important weights to zero + introduce sparsity in the weights by forcing more weights to be zero
- **L2**: cost added proportional to the square of the value of the weights coefficient. Penalize large weights + encourage weights to be small but do not drive them to exactly 0. More used for smaller deep learning models

- **Dropout**: randomly dropping out neurons
    - The output is set to 0
    - No weight update
    - The idea is to introduce noise in the output values to break patterns
    - Learn more robust and general representations

Normalizer

- Batch normalization
    - Good idea to apply in a hidden layer *before* the activation function + put the bias of the layer to false

    - Address the **internal covariance shift** that makes training more difficult and when network is trained the parameters change
    - Idea: normalize the inputs of a layer (mean = 0 and variance = 0.1)
    - Reduce dependence of each layer on the specific distribution of inputs
    - Alleviate vanishing and exploding gradients
    - Reduce the need for other regularization techniques
    - Makes the network more robust to changed
    - Two parameters
        - Gamma: learned scaling factor (initialized as 1)
        - Beta: learned offset factor (initialized as 0)
    - Eliminate the need for dropout
    - Allows to use higher learning rate and to be less careful about initialization
    - Acts as a regularizer

- Feature normalization
    - Preprocessing step
    - Applied to input data before feeding it into a machine
    - MinMaxScaling $x_{new} = \frac{x_{old} - x_{min}}{x_{max} - x_{min}}$

On which hyperparameters would you perform the model selection and why

- **Learning rate**: determine the step size at each iteration
    - How quickly or slowly the model learns from the training data
    - Common to start with a moderate learning rate in the range 0.1 to 0.001
- **Epochs**: number of times the entire dataset is passed in the model
- **Batch size**:
    - Number of sample processed before the model is updated
    - Divide the training dataset into smaller subsets
    - Each mini-batch is a subside of training examples
    - Update the parameters based on the batch
- **Number of layers**
- **Number of neurons**
- **Dropout rate**: fraction of neurons randomly dropped during training to prevent overfitting
- **Weight initialization**
    - Random uniform or normal distribution
    - Xavier/Glorot initialization (Tanh/Sigmoid)
    - He initialization (ReLU)
- **Optimizer**: update the weights of the neural network during training
    - Stochastic gradient descent (SGD)
    - SGD with momentum
    - Adam
    - RMSprop

- Adagard

Convolutional layer

- **Number of filters**: equal to the number of output channels
- **Filter size (kernel)**: size for the convolution window used for operation
  - Can be a tuple or a single element
- **Stride**: step for each sliding, usually equal to 1 or 2, if not specified is equal to the filter size
- **Padding**: *same* or *valid*
  - Same: output dimension = input dimension
  - Valid: no padding applied

Pooling layer

- With pooling the input channel and output channels stays the same
  - Max pooling: take the max value, more common
  - Average pooling: take the average
- **Filter size (F)**: the size of the pooling, either one value or a tuple
- **Stride (S)**: sliding of the tuple
  - Usually:
    - F = 2 and S = 2
    - F = 3 and S = 2
- **Padding**: *same* or *valid*
    - Same: dimension of the output width and height are the same
    - Valid: do not apply padding

- Flattening: convert a matrix in a vector of 1 dimension, used for moving from a CNN to a MLP

How to evaluate the test performance

- During the training part we can set a validation set that gives an estimate of model skill while tuning model's hyperparameters.
- Perform a manual search of the parameters that are difficult to tune in a grid search
- We apply a grid search to find some hyperparameters (like the activation function of the hidden layers). Perform on a small range of hyperparameters (very computationally expensive)
- Check the accuracy and f1_score on the train and test set
  - **Accuracy**: describe how the model performs across all classes. Useful when classes are equally important. Ratio between the number of correct predictions and the total number of predictions
  - **F1_score**: popular metric for classification models that provides accurate results for both balanced and unbalanced dataset. Takes into account both

precision and recall. Is two times the precision multiplied by the recall divided by the sum of precision and recall

- Plot a confusion matrix